

OXYGEN: Unified KV Cache Management for VLA Inference under Multi-Task Parallelism

Xiangyu Li¹ Huaizhi Tang^{2*} Xin Ding^{3*} Weijun Wang¹ Ting Cao^{1†} Yunxin Liu¹

¹Institute for AI Industry Research (AIR), Tsinghua University

²Department of Electronic Engineering, Tsinghua University ³University of Science and Technology of China

*Work done during internships at AIR, Tsinghua University. [†]Corresponding author: Ting Cao (tingcao@mail.tsinghua.edu.cn).

Code: <https://github.com/air-embodied-brain/OxyGen> (Contact: Xiangyu Li, xiangyu.sdlc@foxmail.com)

Embodied AI agents increasingly require parallel execution of multiple tasks, such as manipulation, conversation, and memory construction, from shared observations under distinct time constraints. Recent Mixture-of-Transformers (MoT) Vision-Language-Action Models (VLAs) architecturally support such heterogeneous outputs, yet existing inference systems fail to achieve efficient multi-task parallelism for on-device deployment because of redundant computation and resource contention. We identify isolated KV cache management as the root cause. To address this, we propose *unified KV cache management*, an inference design that treats the KV cache as a first-class shared resource across tasks and over time. This abstraction enables two key optimizations: *cross-task KV sharing* eliminates redundant prefill of shared observations, while *cross-frame continuous batching* decouples variable-length language decoding from fixed-rate action generation across control cycles. We implement this design for $\pi_{0.5}$, a popular MoT VLA, and evaluate it on both NVIDIA GeForce RTX 4090 and Jetson AGX Thor, two representative platforms for on-device VLA inference. OxyGen achieves up to $3.7\times$ speedup over isolated execution, delivering over 200 tokens/s language throughput and 70 Hz action frequency simultaneously without degrading action quality, and we further validate the gains on a real humanoid robot with on-board Jetson AGX Thor.

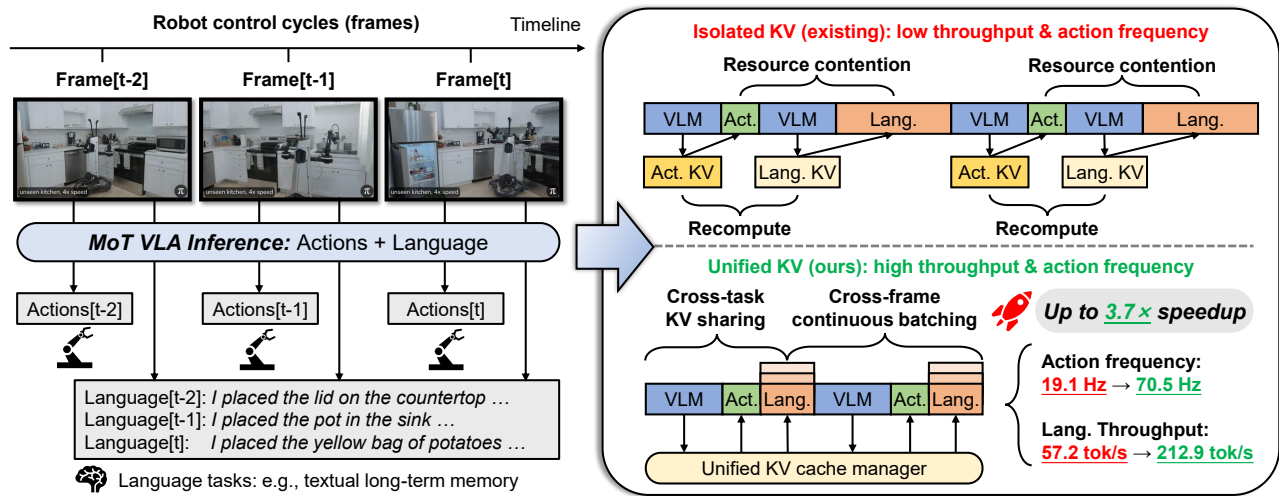


Figure 1 | Left: An example of deploying a Mixture-of-Transformers (MoT) Vision-Language-Action (VLA) model for parallel multi-task inference: from per-frame observations, the VLA generates robot actions within each frame while continuously generating language-based memories across multiple frames [1]. Right: Comparison between two paradigms of MoT VLA inference: existing systems manage the KV cache in isolation, slowing down inference through redundant computation and resource contention; our method adopts *unified KV cache management*, achieving up to $3.7\times$ speedup via cross-task KV sharing and cross-frame continuous batching.

1. Introduction

A long-standing aspiration in embodied AI is to develop agents that, much like humans, coordinate multiple tasks in parallel: conversing while manipulating objects [2–5], or memorizing surroundings while navigating [6–9]. Consider an autonomous home robot such as the one in Fig. 1 [1–3]: while manipulating, it must concurrently memorize environmental changes, narrate progress to the user, and plan ahead to update its schedule. We refer to this setting as *multi-task parallelism*: concurrent execution of temporally independent tasks from shared input, each under its own time constraints.

Recent Mixture-of-Transformers (MoT) [10] Vision-Language-Action Models (VLAs) [11–22] have made strides toward this goal. Conventional VLAs [11, 12, 20, 23] are restricted to the action modality, requiring multi-model inference for multiple tasks (e.g., running a VLA and a VLM concurrently) and making on-device deployment difficult under limited hardware resources. In contrast, recent MoT-VLAs [14, 16, 21] route different outputs to modality-specific experts, enabling a single model to perform both language-based tasks (e.g., planning) and action-based tasks (e.g., manipulation). Yet this architectural multitasking capability does not automatically translate into inference speedups over naive multi-model inference.

We find that existing systems [24–27] perform similarly to naive multi-model inference because they follow an inefficient inference paradigm that we term *isolated execution*: each task is executed through a separate forward pass of the same model, even when tasks share the same input observations (Fig. 1). This leads to two inefficiencies. (1) Redundant computation: the shared observation is encoded repeatedly, producing identical KV cache entries for each task (1.4× slowdown in Section 4.3). (2) Resource contention: even if the KV cache is shared, different tasks compete for limited hardware resources (usually a single GPU on robots) and block each other, despite having different time constraints (2.6× slowdown in Section 4.3). For example, action denoising must complete within each frame (i.e., robot control cycle), while language decoding may span multiple frames. Underlying both issues, we identify a common root cause: existing systems treat each task’s KV cache in isolation, missing opportunities for sharing and coordinated scheduling.

Although KV cache management is well-studied in LLM serving [28, 29], two MoT-specific assumptions break those techniques. (1) *Heterogeneous expert KV semantics*: prefix sharing assumes homogeneous autoregressive readers that extend the cache identically; in MoT VLA, the action expert consumes the prefill cache as read-only context across S denoising steps while the language expert appends new KV entries per decoded token, so naive sharing of a mutable cache can mix language-side appends into the action expert’s read-only cache view. (2) *Frame-bounded language decoding*: continuous batching runs each request uninterrupted to completion, but the physical control loop requires language decoding to advance by a calibrated step budget per frame and resume across frames without recomputation, a pattern existing serving systems do not support.

To address this, we propose *unified KV cache management*, which exposes the KV cache as a *first-class, unified resource managed across tasks and over time*. We realize it in OxyGen, an efficient multi-task inference system for MoT VLAs on robotic platforms, with two optimizations: (1) *cross-task KV sharing* encodes the shared observation once and fans out to per-expert KV views within each frame, eliminating redundant prefill while preserving each expert’s access semantics; (2) *cross-frame continuous batching* groups in-flight language requests across frames into a single decoding batch, advancing each by a calibrated per-frame step budget so action meets its hard deadline while language throughput scales with the active batch.

We implement OxyGen for $\pi_{0.5}$ [14] atop openpi [24], the official inference framework for this widely used MoT VLA (over 10k stars on GitHub). Across the LIBERO, DROID, and ALOHA configurations on NVIDIA GeForce RTX 4090 and Jetson AGX Thor, two representative platforms for on-device VLA inference [13, 30, 31], OxyGen consistently accelerates parallel multi-task inference by up to 3.7 $\times$, sustaining over 200 tokens/s language throughput at 60 Hz action frequency on RTX 4090 and 27 Hz on Jetson AGX Thor. We further validate OxyGen on a real humanoid robot with on-board Jetson AGX Thor, where the action-critical inference path fits within the action-execution window and language-generation latency is largely hidden behind action execution (Section 4.6).

In summary, our contributions are threefold:

- We formulate multi-task parallelism as a target inference scenario for MoT VLAs, and identify isolated KV cache as the root cause of inefficiency in existing systems.
- We propose unified KV cache management, an inference design that treats KV cache as a shared resource across tasks and over time, enabling cross-task KV sharing and cross-frame continuous batching.
- We implement this design for $\pi_{0.5}$ and evaluate it on common robotic GPUs (GeForce RTX 4090 and Jetson AGX Thor), demonstrating up to 3.7 $\times$ speedup in action frequency and language throughput.

2. Related Work

MoT VLAs. Earlier VLA paradigms are restricted to action-only inference: *discrete* VLAs [11, 12] generate actions autoregressively as language tokens, while *continuous* VLAs [13, 15, 17] attach a lightweight diffusion or flow-matching action module for high-frequency control. We instead target the recent MoT VLA family (e.g., $\pi_{0.5}$ [14], WALL-OSS [16], Xiaomi-Robotics-0 [21]), which adopts a Mixture-of-Transformers backbone [10] that routes different output modalities to separate experts while sharing a common Vision-Language Model (VLM) backbone, enabling a single model to jointly produce actions and language (e.g., Chain-of-Thought planning) for long-horizon, dexterous manipulation. Despite this architectural multitasking capability, existing MoT inference systems still execute each task through independent forward passes, gaining no acceleration over naive multi-model inference.

VLA inference optimizations. Existing VLA optimizations target two levels. (1) *Model-level*: VLAs inherit many VLM optimizations, including compression [32–35], token pruning [34, 36–38], layer skipping [39–42], action token reuse [36, 43], KV cache pruning [44], and graph optimization [31]; KV-Efficient VLA [44] in particular selectively activates KV cache at the operator level, whereas OxyGen manages the KV cache at the model level without modifications. (2) *Application-level*: asynchronous pipelines such as RTC [45], SmolVLA [42], and VLA-RAIL [46] extend action chunking [47] to overlap inference with execution and avoid the jerky motion or doubled cost of temporal ensembling, but remain action-only. OxyGen is largely orthogonal to these optimizations: it operates as a scheduling layer targeting multi-task inference, and can be combined with model compression or asynchronous action pipelines in deployment.

KV cache reuse for LLMs. KV cache reuse is widely studied in LLM serving: prefix caching in vLLM [28] and SGLang [29] avoids recomputation when requests share prefix tokens, with extensions to non-prefix [48, 49] and cross-model [50, 51] reuse. These systems target cloud-scale, multi-tenant LLM

inference, where the goal is to maximize aggregate request throughput given abundant compute and high concurrency. On-device multi-task MoT VLA inference is a fundamentally different problem: under tight resource budgets, the system must maximize language throughput subject to a hard per-frame action deadline tied to the physical control loop. As a result, directly applying techniques like prefix caching is insufficient—without a coordinated execution design across modalities, language decoding still blocks action generation and violates the frame budget.

3. Method

We propose OxyGen, an inference system for MoT VLAs that achieves efficient multi-task parallelism through *unified KV cache management*. The key insight is that the KV cache, produced by the shared VLM backbone from a common observation, is a natural locus for both computation reuse and execution coordination.

3.1. Preliminaries and Problem Formulation

MoT VLA inference. We consider a generic multi-task embodied agent based on a MoT VLA, such as $\pi_{0.5}$ [14]. At frame (*i.e.*, control cycle) t , the agent observes $\mathbf{o}_t$, which contains visual inputs for this frame and a language instruction. MoT VLA factorizes inference into prefill with the modality-agnostic backbone and generation with modality-specific experts. The prefill phase is formulated as:

$$\{(\mathbf{h}_{t,l}, \mathbf{K}_{t,l}, \mathbf{V}_{t,l})\}_{l=1}^L = \Theta_{\text{VLM}}(\mathbf{o}_t), \quad \mathcal{K}_t = \{(\mathbf{K}_{t,l}, \mathbf{V}_{t,l})\}_{l=1}^L, \quad (1)$$

where Θ_{VLM} denotes the parameters of the VLM backbone, L is the number of transformer layers in the VLM, $\mathbf{h}_{t,l}, \mathbf{K}_{t,l}, \mathbf{V}_{t,l}$ are hidden states, keys, and values of VLM layer l , and $\mathcal{K}_t$ is the KV cache produced from $\mathbf{o}_t$. Crucially, $\mathcal{K}_t$ is modality-agnostic: it encodes the observation and can be consumed by multiple experts, without committing to a specific output modality.

Given the shared $\mathcal{K}_t$, MoT VLA runs multiple experts independently. In this paper, we focus on two representative experts: an *action expert* that generates an action chunk $\mathbf{A}_t = \{\mathbf{a}_{t,i}\}_{i=1}^H$ (*i.e.*, low-level control commands for a horizon of H), and a *language expert* that generates text tokens $\mathbf{y}_t = \{y_{t,j}\}_{j=1}^N$ (*e.g.*, memory or QA with a maximum token budget of N). Since $\mathcal{K}_t$ encapsulates the visual-language information from $\mathbf{o}_t$, both experts can generate their outputs conditioned on $\mathcal{K}_t$ instead of directly on $\mathbf{o}_t$:

$$p_{\Theta_{\text{Act}}}(\mathbf{A}_t \mid \mathcal{K}_t), \quad p_{\Theta_{\text{Lang}}}(\mathbf{y}_t \mid \mathcal{K}_t), \quad (2)$$

where Θ_{Act} and Θ_{Lang} (usually the language backbone in the VLM) parameterize the action and language distributions, respectively. Concretely, the language expert decodes $\mathbf{y}_t$ autoregressively conditioned on $\mathcal{K}_t$, while the action expert generates the chunk $\mathbf{A}_t$ jointly via S denoising steps over $\mathcal{K}_t$ (implemented as diffusion or flow matching); full formulations are deferred to Section A.1.

Multi-task parallelism with asymmetric deadlines. At each frame t , the agent serves multiple concurrent tasks. We consider action and language tasks for models like $\pi_{0.5}$, where different tasks have asymmetric deadlines. (1) Action $\mathbf{A}_t$ must be generated by a hard per-frame deadline tied to the physical control loop, sustaining a minimum control frequency $f_{\min}$ for smooth robot control (*e.g.*, 50Hz for dexterous manipulation). (2) Language $\mathbf{y}_t$ can be generated under a soft deadline across frames, and we

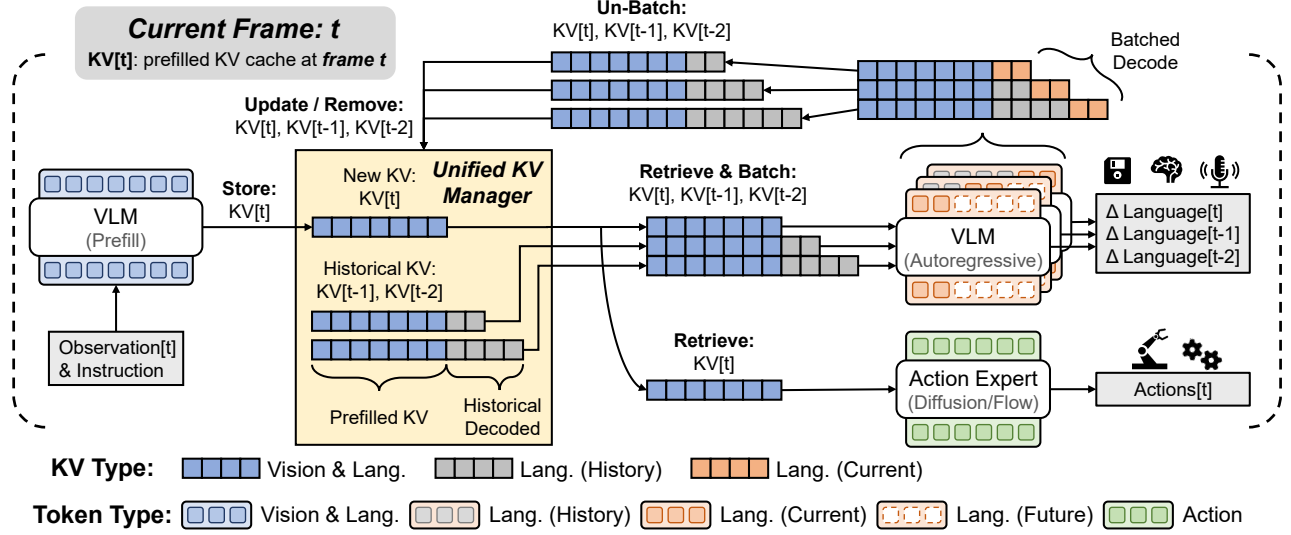


Figure 2 | KV-centric dataflow at frame t with unified KV cache manager. $\text{KV}[t]$ represents KV cache prefilled at frame t (i.e., $\mathcal{K}_t$ defined in Eq. (1)); $\Delta \text{Language}[t]$ represents incremental language tokens in y_t , generated with $\mathcal{K}_t$.

aim to maximize token throughput while satisfying the hard action deadline. Let f and τ denote the actual action frequency and language throughput at steady state. Given the action horizon H , average batch size B , decoding steps per frame k , and end-to-end inference latency T , we have $f = H/T$ and $\tau = Bk/T$, so the application-level objective translates to a model-level form:

$$\max \quad \tau = \frac{Bk}{T} \quad \text{s.t.} \quad f = \frac{H}{T} \geq f_{\min}, \quad (3)$$

which naturally leads to two optimization directions: reducing end-to-end latency and increasing the number of tokens decoded per frame. Existing systems handle these two directions in isolation: language decoding either blocks within a frame and violates the action deadline, or is throttled to a few tokens per frame and sacrifices throughput. In contrast, our method achieves both by treating the KV cache as a unified resource managed across tasks and frames.

3.2. Unified KV Cache Manager

OxyGen abstracts the KV cache as a shared resource across tasks and frames, managed by a *unified KV cache manager* $\mathcal{M}$. The manager supports two key optimizations: **cross-task KV sharing** fans out a single $\mathcal{K}_t$ to heterogeneous action and language experts within frame t , eliminating redundant prefill; **cross-frame continuous batching** groups language requests from different frames into a single decoding batch, decoupling language decoding from the per-frame control loop while preserving the hard action deadline. To support both, $\mathcal{M}$ maintains generation states for each in-flight request, tracking their KV caches and decoded tokens.

Resumable generation state. Cross-task KV sharing within a frame is straightforward: all experts at frame t consume the same prefill cache $\mathcal{K}_t$. Interrupting and resuming language generation across

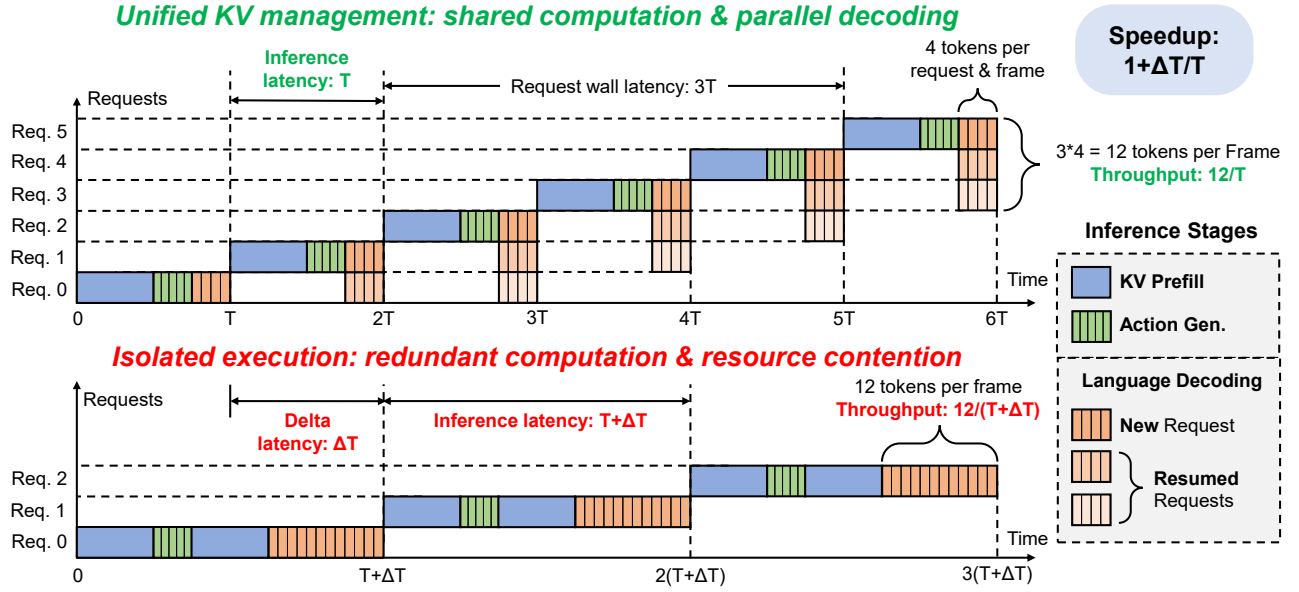


Figure 3 | Timeline comparison of OxyGen vs. isolated execution (baseline), with an example workload of $N = 12$ total tokens per request. After the initial warmup, OxyGen steadily advances $B = 3$ parallel requests to produce $k = 4$ tokens per request per frame, significantly reducing the end-to-end inference latency per frame, and increasing both action frequency and language throughput, all by a factor of $1 + \Delta T/T$.

frames, however, requires representing each request by an *incremental state*:

$$\sigma_t = (\mathcal{K}_t, \mathbf{y}_t, \delta_t), \quad (4)$$

where $\mathcal{K}_t$ is the KV cache for the request initiated at frame t (initially the prefill cache from Eq. (1)), then extended with decoded token KVs as generation progresses), $\mathbf{y}_t$ is the token buffer storing generated tokens, and $\delta_t \in \{0, 1\}$ is a termination flag (set to 1 when EOS is emitted or maximum length N is reached). Crucially, σ_t contains all necessary context to resume autoregressive language generation without recomputation. The manager $\mathcal{M}$ exposes standard CRUD operations (STORE, RETRIEVE, UPDATE, REMOVE) on per-request states σ_t , and maintains the set of active requests $\mathcal{R} = \{r_{t_1}, \dots, r_{t_m}\}$ whose generation has not yet terminated. For parallel decoding, $\mathcal{M}$ stacks the active states along the batch dimension into a batched state $\hat{\sigma}$, so that the VLM advances all $m = |\mathcal{R}|$ requests in a single forward pass; the formal interface and definition of $\hat{\sigma}$ are given in Section A.2.

Per-frame execution flow. The two optimizations compose into a two-stage per-frame execution. First, the system runs prefill once on the new observation $\mathbf{o}_t$ to obtain $\mathcal{K}_t$ (Eq. (1)); the manager fans this cache out to the action expert (which denoises $\mathbf{A}_t$ over S steps) and to a freshly initialized language state σ_t stored under request ID r_t , eliminating the redundant prefill required by isolated execution. Second, r_t joins the active set $\mathcal{R}$, and $\mathcal{M}$ retrieves and batches all m active states into $\hat{\sigma}$ for k steps of joint autoregressive decoding; finished requests are then evicted and the rest are persisted with their incremental tokens $\mathbf{y}_{t_i}$ for the next frame. The per-frame budget k is calibrated offline from the target frame rate and hardware so that batched decoding over a typical active batch size fits within the

remaining time after action denoising, preserving $f_{\min}$ while allowing language throughput to scale with the active batch size. Algorithm 1 in Section A.3 formalizes this flow. Fig. 3 illustrates the combined effect: cross-task KV sharing provides the initial speedup by eliminating redundant prefill (the bottleneck for long contexts or short decoding), while cross-frame continuous batching further reduces per-frame latency as decoding length increases.

Discussion: generalization to richer settings. We instantiate OxyGen with two experts and standard autoregressive language decoding to match $\pi_{0.5}$, but the manager interface does not depend on either choice. We discuss two natural extensions. **(i) More experts.** For settings with additional concurrent tasks, as discussed in Section 1, $\mathcal{M}$ treats experts as opaque consumers of $\mathcal{K}_t$, so fanning out to $K > 2$ experts requires no change to $\hat{\delta}$. When several of the extra tasks share a language backbone, a natural realization is a shared VLM with per-task LoRA adapters, where existing techniques such as cross-model KV transfer [50, 51] and multi-LoRA serving [52, 53] can be plugged into our framework. **(ii) Parallel decoding.** The $\pi_{0.5}$ implementation is vanilla token-by-token, but the resumable state $\sigma_t = (\mathcal{K}_t, \mathbf{y}_t, \delta_t)$ does not assume one token per forward pass: it would remain a valid anchor under speculative or parallel decoding schemes [54–56].

4. Experiments

4.1. Experimental Setup

4.1.1. Models, configurations, and hardware.

We evaluate OxyGen with $\pi_{0.5}$ [14] on both NVIDIA GeForce RTX 4090 and Jetson AGX Thor, two representative platforms for on-device VLA inference [13, 30, 31]. We evaluate inference speed under three representative configurations derived from LIBERO [57], DROID [58], and ALOHA [47], matching their input observation specifications (*e.g.*, camera views and resolution) and control dimensionality rather than full rollout behavior. Specifically, we evaluate task success rate on LIBERO with the officially released $\pi_{0.5}$ -LIBERO checkpoint, demonstrating that OxyGen does not degrade action quality while accelerating inference. This section mainly reports results on LIBERO; full results are deferred to §B.1.

4.1.2. Baselines.

We compare OxyGen against openpi [24], the official inference framework for $\pi_{0.5}$ (over 10k stars on GitHub), running in two execution modes:

Sequential isolated execution (the main baseline, denoted as “Baseline”), is the standard inference paradigm of existing systems [24–27]: each task (action and language generation) runs independently and sequentially within each frame. Since openpi does not release code for the language generation described in the $\pi_{0.5}$ paper, we implement this baseline following a community reproduction ¹.

Parallel isolated execution (an additional baseline, denoted as “Parallel”) is a straightforward way to parallelize multi-task inference: each task runs on an individual process in parallel, sharing one

¹https://github.com/BrunoFANG1/openpi_subtask_generation.

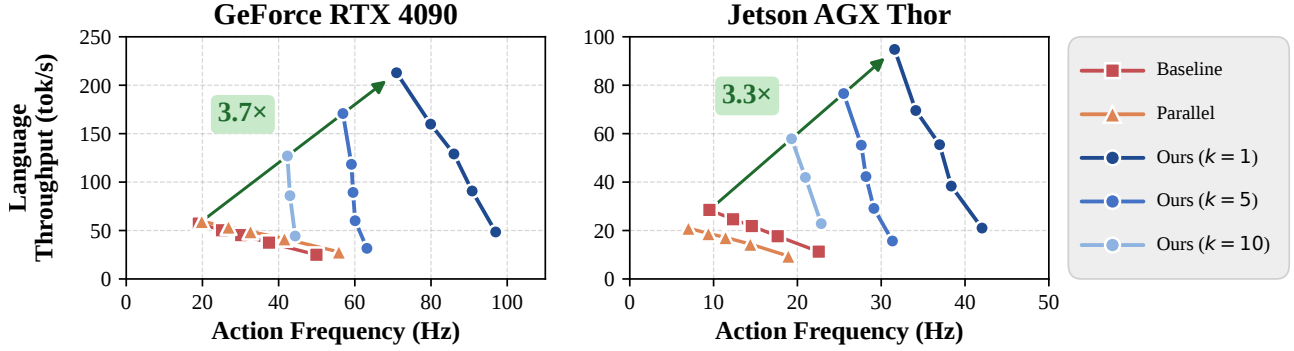


Figure 4 | Action-frequency vs. language-throughput tradeoff on the LIBERO configuration. We sweep decoding steps per frame $k \in \{1, 5, 10\}$ and maximum total decoding steps $N \in \{5, 10, 15, 20, 30\}$; each line fixes k and varies N (markers are individual settings). Each baseline curve must trade one axis for the other as N grows; OxyGen pushes the Pareto frontier outward by up to $3.7\times$.

GPU. We implement this for openpi via CUDA Multi-Process Service (MPS). Results show that this naive parallelization provides very limited speedup from the main baseline.

4.1.3. Metrics.

We measure the following metrics to capture both inference speed and deployment efficiency. **Action frequency** (Hz): number of actions generated per second, determining the smoothness of robot control. Following default openpi settings, we set the action horizon $H = 10$ and denoising steps $S = 10$ during evaluation, unless otherwise specified. **Language throughput** (tokens/s): number of language tokens generated per second, reflecting the speed of language generation. **Average batch size**: average number of concurrent requests during language decoding, reflecting the parallelism of language generation. **Memory** (GB), **Power** (W), and **Energy/Request** (mJ): metrics reflecting the system’s memory and energy efficiency for on-device deployment.

4.2. End-to-End Results

We compare OxyGen against both sequential and parallel isolated execution (denoted as “Baseline” and “Parallel”) across settings that vary the decoding steps per frame $k \in \{1, 5, 10\}$ and the maximum total decoding steps N . One new observation and request arrive each frame. Fig. 4 sweeps the joint action-frequency / language-throughput operating space on the LIBERO configuration; each curve fixes k and varies N , tracing how each system trades off the two axes as the language workload grows. Fig. 5 reports the action-frequency speedup ratio over the baseline as we vary k at fixed $N = 30$.

Key observations. (1) OxyGen extends the action-frequency / language-throughput Pareto frontier: where each baseline curve must sacrifice one axis to improve the other as N grows, OxyGen simultaneously achieves $1.2\text{--}3.7\times$ speedup on both, demonstrating that our method coordinates the heterogeneous generation tasks rather than trading one for the other. (2) OxyGen achieves higher speedup with larger total decoding steps N and smaller decoding steps per frame k , which implies a larger average batch size $B = N/k$. With a large batch size, cross-frame continuous batching better utilizes hardware parallelism

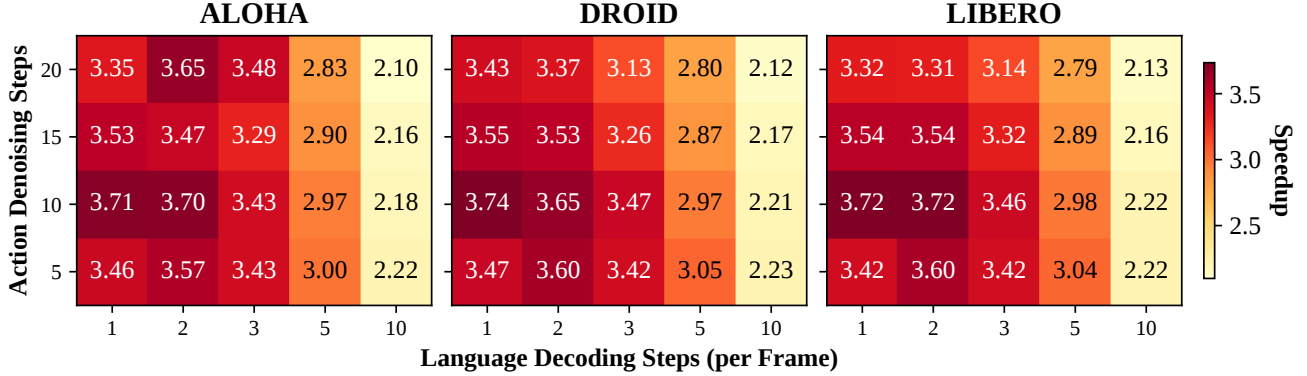


Figure 5 | Speedup ratio for action frequency of OxyGen vs. baseline across LIBERO, ALOHA, and DROID configurations (subplots), action denoising steps S (y-axis), and decoding steps per frame k (x-axis), with total decoding steps fixed at $N = 30$. The speedup is primarily affected by k , while the configuration choice and S have smaller impact.

and achieves significant speedup. (3) Naive MPS parallelization provides modest improvement over the sequential baseline, demonstrating that simply running tasks in parallel without eliminating redundant computation is insufficient.

4.3. Ablation Study

We ablate the two core optimizations to understand their individual contributions. Starting from sequential isolated execution (“Baseline”) and naive multi-process parallelization (“Parallel”), we incrementally enable (A) cross-task KV sharing (“Ours w/o Batching”) and (B) cross-frame continuous batching (“Ours”), measuring action frequency. For a more comprehensive view, we also measure a single-frame decoding oracle (“Batching Upper Bound”) that decodes only the per-frame budget of k language tokens and drops the remaining $N - k$ tokens, so no language request carries over to later frames. This estimates the action-frequency ceiling when cross-frame language backlog is removed. Fig. 6 shows the action frequency on LIBERO across language decoding steps (5 steps per frame) on both GeForce RTX 4090 and Jetson AGX Thor.

Key observations. (1) Cross-task KV sharing provides the initial speedup (1.4× for short decoding steps without batching on RTX 4090), by eliminating redundant prefill of the same observation for different tasks. (2) Cross-frame continuous batching is crucial for scenarios with long decoding steps (≥ 10), where it maintains nearly constant action frequency (around 60 Hz on RTX 4090 and 27 Hz on Jetson AGX Thor). In contrast, other settings without batching degrade significantly as decoding steps increase (e.g., baseline on RTX 4090 drops from 49.9 Hz to 19.1 Hz, by 2.6×). The gap between OxyGen and the oracle upper bound indicates the overhead of cross-frame scheduling and remains modest on both platforms.

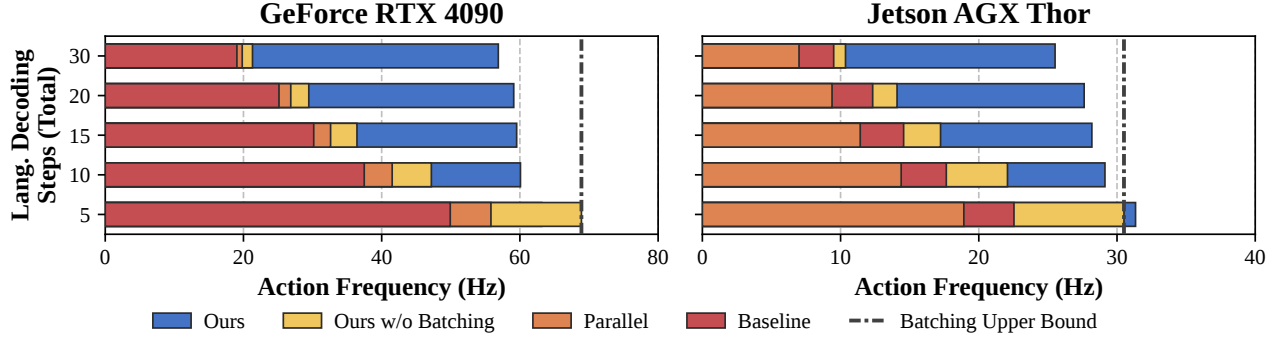


Figure 6 | Ablation of action frequency on LIBERO across language decoding steps (5 steps per frame), measured on GeForce RTX 4090 (left) and Jetson AGX Thor (right). Cross-task KV sharing (“Ours w/o Batching”) provides the initial speedup for short decoding steps, while cross-frame continuous batching (“Ours”) maintains high action frequency as decoding steps increase, on both platforms. The dash-dotted line marks the single-frame decoding oracle upper bound.

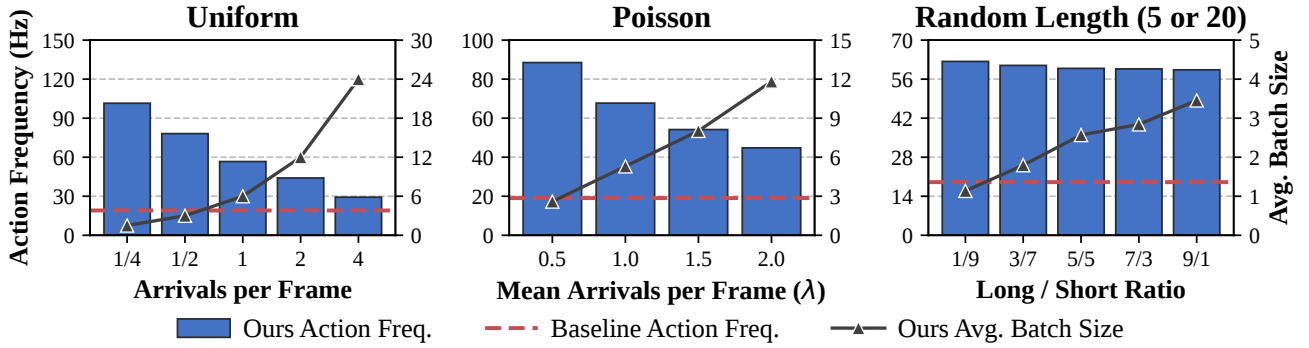


Figure 7 | Action frequency (per request) and average batch size of OxyGen under different request arrival patterns.

4.4. Workload Generalization

Beyond the common workload of one new observation per frame, we demonstrate OxyGen’s ability to generalize to additional workloads, including uniform arrivals with varying requests per frame, Poisson arrivals with varying intensity λ (mean arrivals per frame), and random-length requests with different ratios of short and long decoding lengths. Fig. 7 shows frame latency and average batch size across all patterns. The baseline is for reference only as it always executes requests sequentially without batching.

Key observations. (1) Across all workloads, OxyGen maintains higher action frequency than the baseline. (2) Under uniform and Poisson arrivals, a higher arrival rate increases the average batch size and thus language throughput, but reduces action frequency per request. This is expected because when more than one observation requires prefill in a single frame, prefill cost dominates model inference, degrading the action frequency of each request. However, the total actions per second still increases (e.g., by 4.4 $\times$ in “Uniform”), indicating higher hardware utilization. (3) Under random request lengths,



Stage	Baseline (ms)	OxyGen (ms)
Video stream	9.4	-
Prefill & denoise	207.5	198.0
Lang. generation	822.3	195.4
Action comm.	0.6	-
Action exec.	<u>333.0</u>	-

Figure 8 | Real humanoid robot with on-board Jetson AGX Thor (left) and per-frame breakdown (ms) measured on board (right). OxyGen cuts language generation by 4.2 $\times$ (822.3 $\rightarrow$ 195.4 ms).

OxyGen maintains a constant action frequency per request while flexibly handling different numbers of concurrent requests.

4.5. Action Quality and Resource Efficiency

OxyGen optimizes the VLA inference pipeline without degrading action quality: with the $\pi_{0.5}$ -LIBERO checkpoint, it reproduces the openpi-reported success rates within $\pm 0.8\%$ across LIBERO- $\{\text{Spatial, Long, Goal, 10}\}$ (Table 1 in Section B.2). On the same hardware, OxyGen is also resource-efficient: it adds modest memory overhead (15%) while reducing average power by up to 47% and energy per request by up to 78%, owing to fewer memory accesses to VLM weights during batched decoding. Naive parallelization, in contrast, nearly doubles peak memory and even increases energy per request by 3% because of redundant computation and memory bandwidth contention (Table 2 in Section B.3).

4.6. Real-Robot Deployment

To verify that the speedups translate beyond simulation, we deploy OxyGen on a real humanoid robot with an on-board Jetson AGX Thor module, running $\pi_{0.5}$ in a multi-task setting that interleaves manipulation with concurrent language generation. We use 3-way RGB observations at 224×224 , action horizon $H = 10$, denoising steps $S = 10$, and total decoding steps $N = 30$ at $k = 5$ steps/frame. Fig. 8 shows the platform and the measured per-frame breakdown: baseline on-device inference (1030 ms) far exceeds the 333 ms action-execution window and blocks the next control cycle. OxyGen reduces total inference to 393 ms; more importantly, the action-critical prefill+denoise stage takes 198 ms and fits within the execution window, while the remaining 195 ms of language generation runs after the action is produced and is largely hidden behind action execution.

5. Conclusion

We present OxyGen, an efficient inference system for MoT VLAs under multi-task parallelism. By proposing unified KV cache management, a novel inference design that treats the KV cache as a shared resource across tasks and over time, we enable cross-task KV sharing and cross-frame continuous batching. We implement OxyGen for $\pi_{0.5}$ and achieve up to 3.7 $\times$ speedup on NVIDIA GeForce RTX 4090 and Jetson

AGX Thor, two representative platforms for on-device VLA inference, demonstrating that unified KV cache management is critical for efficient embodied agents with multi-task capabilities.

Limitations. Our empirical evaluation focuses on one existing MoT VLA ($\pi_{0.5}$) with two experts (action and language); generalization to other MoT backbones and richer multi-expert configurations is discussed in Section 3.2 but left to future work.

Broader impacts. OxyGen improves the efficiency of on-device VLA inference, lowering the compute and energy cost of deploying multi-task embodied agents on commodity robotic hardware. This can improve access to robotic research and reduce the energy footprint of deployment.

References

- [1] Marcel Torne, Karl Pertsch, Homer Walke, Kyle Vedder, Suraj Nair, Brian Ichter, Allen Z. Ren, Haohuan Wang, Jiaming Tang, Kyle Stachowicz, Karan Dhabalia, Michael Equi, Quan Vuong, Jost Tobias Springenberg, Sergey Levine, Chelsea Finn, and Danny Driess. Mem: Multi-scale embodied memory for vision language action models. 2025. URL <https://www.pi.website/download/Mem.pdf>. Technical report, Physical Intelligence.
- [2] Figure AI. Introducing Figure 03: Third Generation Humanoid Robot. <https://www.figure.ai/news/introducing-figure-03>, October 2025. Accessed: 2026-02-14.
- [3] 1X Technologies. NEO Home Robot: The World’s First Consumer-Ready Humanoid Robot. <https://www.1x.tech/neo>, 2026. Accessed: 2026-02-14.
- [4] Dong Won Lee, Sarah Gillet, Louis-Philippe Morency, Cynthia Breazeal, and Hae Won Park. A modern system recipe for situated embodied human-robot conversation with real-time multimodal llms and tool-calling. *arXiv preprint arXiv:2602.04157*, 2026.
- [5] Lucy Xiaoyang Shi, Brian Ichter, Michael Equi, Liyiming Ke, Karl Pertsch, Quan Vuong, James Tanner, Anna Walling, Haohuan Wang, Niccolo Fusai, et al. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. *arXiv preprint arXiv:2502.19417*, 2025.
- [6] Abrar Anwar, John Welsh, Joydeep Biswas, Soha Pouya, and Yan Chang. Remembr: Building and reasoning over long-horizon spatio-temporal memory for robot navigation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2838–2845. IEEE, 2025.
- [7] Abhinav Rajvanshi, Karan Sikka, Xiao Lin, Bhoram Lee, Han-Pang Chiu, and Alvaro Velasquez. Saynav: Grounding large language models for dynamic planning to navigation in new environments. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 34, pages 464–474, 2024.
- [8] Qiao Gu, Ali Kuwajerwala, Sacha Morin, Krishna Murthy Jatavallabhula, Bipasha Sen, Aditya Agarwal, Corban Rivera, William Paul, Kirsty Ellis, Rama Chellappa, et al. Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5021–5028. IEEE, 2024.

-
- [9] Nuri Kim, Obin Kwon, Hwiyeon Yoo, Yunho Choi, Jeongho Park, and Songhwai Oh. Topological semantic graph memory for image-goal navigation. In *Conference on Robot Learning*, pages 393–402. PMLR, 2023.
- [10] Weixin Liang, Lili Yu, Liang Luo, Srinivasan Iyer, Ning Dong, Chunting Zhou, Gargi Ghosh, Mike Lewis, Wen-tau Yih, Luke Zettlemoyer, et al. Mixture-of-transformers: A sparse and scalable architecture for multi-modal foundation models. *arXiv preprint arXiv:2411.04996*, 2024.
- [11] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [12] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [13] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [14] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [15] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [16] Andy Zhai, Brae Liu, Bruno Fang, Chalse Cai, Ellie Ma, Ethan Yin, Hao Wang, Hugo Zhou, James Wang, Lights Shi, et al. Igniting vlms toward the embodied space. *arXiv preprint arXiv:2509.11766*, 2025.
- [17] Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024.
- [18] Tao Jiang, Tianyuan Yuan, Yicheng Liu, Chenhao Lu, Jianning Cui, Xiao Liu, Shuiqi Cheng, Jiyang Gao, Huazhe Xu, and Hang Zhao. Galaxea open-world dataset and g0 dual-system vla model. *arXiv preprint arXiv:2509.00576*, 2025.
- [19] Junjie Wen, Yichen Zhu, Jinming Li, Zhibin Tang, Chaomin Shen, and Feifei Feng. Dexvla: Vision-language model with plug-in diffusion expert for general robot control. *arXiv preprint arXiv:2502.05855*, 2025.
- [20] Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. Univla: Learning to act anywhere with task-centric latent actions. *arXiv preprint arXiv:2505.06111*, 2025.
- [21] Xiaomi Robotics. Xiaomi-robotics-0: An open-sourced vision-language-action model with real-time execution. *arXiv preprint*, 2026.
-

-
- [22] Jun Cen, Siteng Huang, Yuqian Yuan, Kehan Li, Hangjie Yuan, Chaohui Yu, Yuming Jiang, Jiayan Guo, Xin Li, Hao Luo, et al. Rynnvla-002: A unified vision-language-action and world model. *arXiv preprint arXiv:2511.17502*, 2025.
 - [23] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
 - [24] Physical Intelligence. openpi, 2025. URL <https://github.com/Physical-Intelligence/openpi>. Apache-2.0 license.
 - [25] X-Square-Robot. wall-x: Building general-purpose robots based on embodied foundation model, 2025. URL <https://github.com/X-Square-Robot/wall-x>.
 - [26] OpenGalaxea. Galaxeavla: Galaxea’s open-source vla repository, 2025. URL <https://github.com/OpenGalaxea/GalaxeaVLA>. G0 PLUS Community License (Non-Commercial).
 - [27] Xiaomi Robotics. Xiaomi-robotics-0: An open-sourced vision-language-action model with real-time inference, 2026. URL <https://github.com/XiaomiRobotics/Xiaomi-Robotics-0>. Apache-2.0 license.
 - [28] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.
 - [29] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37: 62557–62583, 2024.
 - [30] Wenqi Jiang, Jason Clemons, Karu Sankaralingam, and Christos Kozyrakis. How fast can i run my vla? demystifying vla inference performance with vla-perf. *arXiv preprint arXiv:2602.18397*, 2026.
 - [31] Yunchao Ma, Yizhuang Zhou, Yunhuan Yang, Tiancai Wang, and Haoqiang Fan. Running vlas at real-time speed. *arXiv preprint arXiv:2510.26742*, 2025.
 - [32] Hongyu Wang, Chuyan Xiong, Ruiping Wang, and Xilin Chen. Bitvla: 1-bit vision-language-action models for robotics manipulation. *arXiv preprint arXiv:2506.07530*, 2025.
 - [33] Hengyu Fang, Yijiang Liu, Yuan Du, Li Du, and Huanrui Yang. Sqap-vla: A synergistic quantization-aware pruning framework for high-performance vision-language-action models. *arXiv preprint arXiv:2509.09090*, 2025.
 - [34] Yantai Yang, Yuhao Wang, Zichen Wen, Luo Zhongwei, Chang Zou, Zhipeng Zhang, Chuan Wen, and Linfeng Zhang. Efficientvla: Training-free acceleration and compression for vision-language-action models. *arXiv preprint arXiv:2506.10100*, 2025.
 - [35] Seongmin Park, Hyungmin Kim, Wonseok Jeon, Juyoung Yang, Byeongwook Jeon, Yoonseon Oh, and Jungwook Choi. Quantization-aware imitation-learning for resource-efficient robotic control. *arXiv preprint arXiv:2412.01034*, 2024.
-

-
- [36] Xudong Tan, Yaoxin Yang, Peng Ye, Jialin Zheng, Bizhe Bai, Xinyi Wang, Jia Hao, and Tao Chen. Think twice, act once: Token-aware compression and action reuse for efficient inference in vision-language-action models. *arXiv preprint arXiv:2505.21200*, 2025.
 - [37] Ye Li, Yuan Meng, Zewen Sun, Kangye Ji, Chen Tang, Jiajun Fan, Xinzhu Ma, Shutao Xia, Zhi Wang, and Wenwu Zhu. Sp-vla: A joint model scheduling and token pruning approach for vla model acceleration. *arXiv preprint arXiv:2506.12723*, 2025.
 - [38] Titong Jiang, Xuefeng Jiang, Yuan Ma, Xin Wen, Bailin Li, Kun Zhan, Peng Jia, Yahui Liu, Sheng Sun, and Xianpeng Lang. The better you learn, the smarter you prune: Towards efficient vision-language-action models via differentiable token pruning. *arXiv preprint arXiv:2509.12594*, 2025.
 - [39] Rongyu Zhang, Menghang Dong, Yuan Zhang, Liang Heng, Xiaowei Chi, Gaole Dai, Li Du, Yuan Du, and Shanghang Zhang. Mole-vla: Dynamic layer-skipping vision language action model via mixture-of-layers for efficient robot manipulation. *arXiv preprint arXiv:2503.20384*, 2025.
 - [40] Yang Yue, Yulin Wang, Bingyi Kang, Yizeng Han, Shenzhi Wang, Shiji Song, Jiashi Feng, and Gao Huang. Deer-vla: Dynamic inference of multimodal large language models for efficient robot execution. *Advances in Neural Information Processing Systems*, 37:56619–56643, 2024.
 - [41] Moritz Reuss, Hongyi Zhou, Marcel Rühle, Ömer Erdiñç Yağmurlu, Fabian Otto, and Rudolf Lioutikov. Flower: Democratizing generalist robot policies with efficient vision-language-action flow policies. *arXiv preprint arXiv:2509.04996*, 2025.
 - [42] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
 - [43] Siyu Xu, Yunke Wang, Chenghao Xia, Dihao Zhu, Tao Huang, and Chang Xu. Vla-cache: Efficient vision-language-action manipulation via adaptive token caching. *arXiv preprint arXiv:2502.02175*, 2025.
 - [44] Wanshun Xu, Long Zhuang, and Lianlei Shan. Kv-efficient vla: A method to speed up vision language models with rnn-gated chunked kv cache. *arXiv preprint arXiv:2509.21354*, 2025.
 - [45] Kevin Black, Manuel Y Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. *arXiv preprint arXiv:2506.07339*, 2025.
 - [46] Yongsheng Zhao, Lei Zhao, Baoping Cheng, Gongxin Yao, Xuanzhang Wen, and Han Gao. Vla-rail: A real-time asynchronous inference linker for vla models and robots. *arXiv preprint arXiv:2512.24673*, 2025.
 - [47] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
 - [48] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the twentieth European conference on computer systems*, pages 94–109, 2025.
-

-
- [49] Huan Yang, Renji Zhang, Mingzhe Huang, Weijun Wang, Yin Tang, Yuanchun Li, Yunxin Liu, and Deyu Zhang. Kvshare: An llm service system with efficient and effective multi-tenant kv cache reuse. *arXiv preprint arXiv:2503.16525*, 2025.
 - [50] Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. Cache-to-cache: Direct semantic communication between large language models. *arXiv preprint arXiv:2510.03215*, 2025.
 - [51] Yuhan Liu, Yuyang Huang, Jiayi Yao, Shaoting Feng, Zhuohan Gu, Kuntai Du, Hanchen Li, Yihua Cheng, Junchen Jiang, Shan Lu, et al. Droidspeak: Kv cache sharing for cross-llm communication and multi-llm serving. *arXiv preprint arXiv:2411.02820*, 2024.
 - [52] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. S-LoRA: Serving thousands of concurrent LoRA adapters. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
 - [53] Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. Punica: Multi-tenant LoRA serving. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
 - [54] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*, 2024.
 - [55] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning (ICML)*, 2024.
 - [56] Lefan Zhang, Xiaodan Wang, Yanhua Huang, and Ruiwen Xu. Learning harmonized representations for speculative sampling. In *International Conference on Learning Representations (ICLR)*, 2025.
 - [57] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
 - [58] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.

A. Method Details

This section records the formal definitions and algorithm deferred from Section 3.

A.1. Generation Formulations for Language and Action Experts

Given the shared $\mathcal{K}_t$ from Eq. (1), the language expert generates text tokens $\mathbf{y}_t$ autoregressively, where each token depends on all previous tokens:

$$p_{\Theta_{\text{Lang}}}(\mathbf{y}_t \mid \mathcal{K}_t) = \prod_{j=1}^N p_{\Theta_{\text{Lang}}}(\mathbf{y}_{t,j} \mid \mathbf{y}_{t,1:j-1}, \mathcal{K}_t), \quad (5)$$

while the action expert generates the entire action chunk $\mathbf{A}_t = \{\mathbf{a}_{t,i}\}_{i=1}^H$ jointly through an iterative denoising process over S steps:

$$\mathbf{A}_t = \text{Denoise}_{\Theta_{\text{Act}}}^{(S)}(\boldsymbol{\epsilon}, \mathcal{K}_t), \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (6)$$

where each denoising step conditions on the shared KV cache $\mathcal{K}_t$. This process can be implemented via diffusion or flow matching.

A.2. Unified KV Cache Manager: Interface Details

We expand the manager interfaces summarized in Section 3.2.

Interface for state persistence. The manager $\mathcal{M}$ exposes four core operations to persist and retrieve generation states σ_t defined in Eq. (4):

$\mathcal{M}.\text{STORE}(\sigma_t) \rightarrow r_t$	persists state σ_t and returns request ID r_t
$\mathcal{M}.\text{RETRIEVE}(r_t) \rightarrow \sigma_t$	fetches state by ID r_t
$\mathcal{M}.\text{UPDATE}(r_t, \sigma'_t) \rightarrow \emptyset$	replaces existing state of ID r_t with σ'_t
$\mathcal{M}.\text{REMOVE}(r_t) \rightarrow \emptyset$	evicts finished request state by ID r_t

Request IDs are assigned sequentially: r_t is the total number of requests created so far. In simple scenarios with one new request per frame (as in Fig. 3), we have $r_t = t$. The updated state $\sigma'_t = (\mathcal{K}'_t, \mathbf{y}'_t, \delta'_t)$ reflects incremental progress after decoding k tokens: $\mathcal{K}'_t$ extends $\mathcal{K}_t$ with KVs from newly decoded tokens, $\mathbf{y}'_t$ appends these k tokens to $\mathbf{y}_t$, and δ'_t is set to 1 if generation terminates (EOS emitted or length N reached). These operations enable correct resumable generation: requests can be interrupted and resumed across frames without recomputation.

Interface for batched decoding. At any given time, the manager maintains a set of active requests $\mathcal{R} = \{r_{t_1}, r_{t_2}, \dots, r_{t_m}\}$, where each r_{t_i} is the request ID of a request initiated at frame t_i that has not yet terminated ($\delta_{t_i} = 0$). When a new request r_t is created at frame t , it is immediately added to $\mathcal{R}$ and participates in batched decoding. To enable efficient parallel decoding across all $m = |\mathcal{R}|$ active requests, the manager defines a *batched state*:

$$\hat{\sigma} = (\hat{\mathcal{K}}, \hat{\mathbf{y}}, \hat{\boldsymbol{\delta}}), \quad (7)$$

where $\hat{\mathcal{K}} = \{(\hat{\mathbf{K}}_l, \hat{\mathbf{V}}_l)\}_{l=1}^L$ stacks KV caches from $\{\mathcal{K}_{t_i}\}_{i=1}^m$ along the batch dimension at each layer l , $\hat{\mathbf{y}} = [\mathbf{y}_{t_1}; \mathbf{y}_{t_2}; \dots; \mathbf{y}_{t_m}]$ concatenates token buffers, and $\hat{\boldsymbol{\delta}} = [\delta_{t_1}, \delta_{t_2}, \dots, \delta_{t_m}]$ collects termination flags. For

Algorithm 1: Per-Frame Execution Flow

Input: Frame index t , new observation $\mathbf{o}_t$, active request IDs $\mathcal{R} = \{r_{t_i}\}_{i=1}^{m-1}$, manager $\mathcal{M}$, per-frame decoding step k , action denoising steps S
Output: Actions $\mathbf{A}_t$, updated language tokens $\{y_{t_i}\}_{i=1}^m$.

```
// New request: generate actions and initialize language
1  $\mathcal{K}_t \leftarrow \text{PREFILL}(\mathbf{o}_t);$ 
2  $\mathbf{A}_t \leftarrow \text{ACTIONDENOISE}(\mathcal{K}_t, S);$ 
3  $\sigma_t \leftarrow \text{INITSTATE}(\mathcal{K}_t);$ 
4  $r_t \leftarrow \mathcal{M}.\text{STORE}(\sigma_t);$ 

// Batched decode: advance all  $m$  requests by  $k$  tokens
5  $\mathcal{R} \leftarrow \mathcal{R} \cup \{r_t\};$  // Add new request to active set, now  $|\mathcal{R}| = m$ 
6  $\{\sigma_{t_i}\}_{i=1}^m \leftarrow \{\mathcal{M}.\text{RETRIEVE}(r_{t_i}) \mid r_{t_i} \in \mathcal{R}\};$ 
7  $\hat{\sigma} \leftarrow \text{BATCH}(\{\sigma_{t_i}\}_{i=1}^m);$ 
8  $\hat{\sigma}' \leftarrow \text{BATCHEDLANGUAGEDECODE}(\hat{\sigma}, k);$ 
9  $\{\sigma'_{t_i}\}_{i=1}^m \leftarrow \text{UNBATCH}(\hat{\sigma}');$ 

// Extract language outputs and update or evict requests
10 foreach  $i = 1$  to  $m$  do
11    $y_{t_i} \leftarrow \sigma'_{t_i}.\mathcal{Y};$  // Get updated tokens per request
12   if  $\sigma'_{t_i}.\delta = 1$  then  $\mathcal{M}.\text{REMOVE}(r_{t_i});$ 
13   else  $\mathcal{M}.\text{UPDATE}(r_{t_i}, \sigma'_{t_i});$ 
```

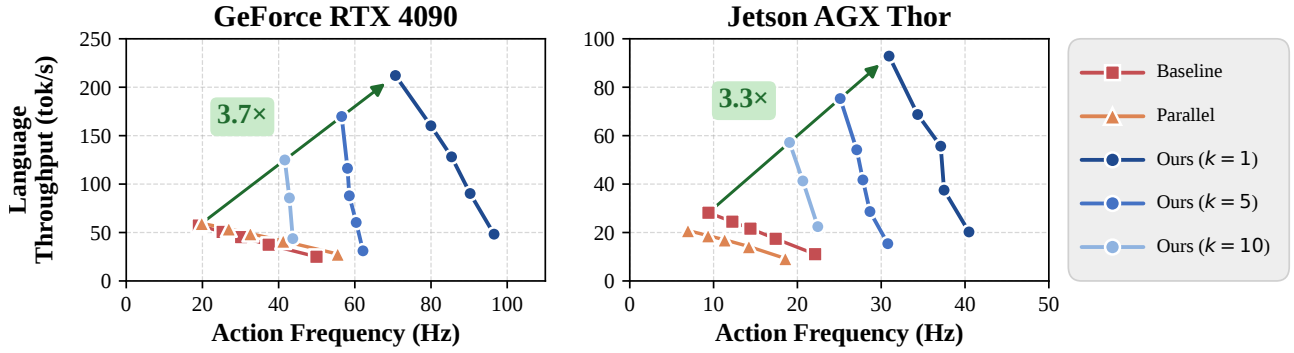
newly created requests at the current frame, their token buffers are initially empty. The manager provides two operations to convert between individual and batched states:

$$\begin{aligned} \mathcal{M}.\text{BATCH}(\{\sigma_{t_i}\}_{i=1}^m) &\rightarrow \hat{\sigma} && \text{concatenates states into batched state } \hat{\sigma} \\ \mathcal{M}.\text{UNBATCH}(\hat{\sigma}') &\rightarrow \{\sigma'_{t_i}\}_{i=1}^m && \text{splits batched state into individual states} \end{aligned}$$

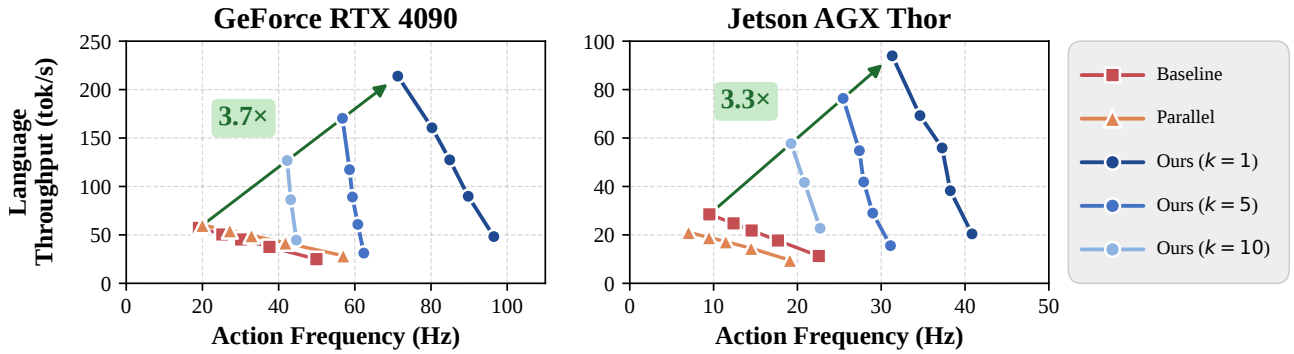
The batched state $\hat{\sigma}$ enables the VLM to perform autoregressive decoding (Eq. (5)) on all m requests in parallel: at each decoding step, the VLM consumes $\mathcal{K}$ as the attention context and $\hat{\mathbf{y}}$ as the token buffer for history tokens, generating the next token for each request simultaneously in a single forward pass. This amortizes the decode cost over multiple requests and achieves significant speedup on modern accelerators.

A.3. Per-Frame Execution Algorithm

Algorithm 1 formalizes the per-frame execution flow described in Section 3.2. At each frame, the system processes one new observation (spawning a language generation request and producing actions) while continuing $m - 1$ in-flight language requests from previous frames, resulting in m total active requests processed in parallel. The algorithm proceeds in two stages: lines 1–4 run prefill once on the new observation and fan the shared KV cache $\mathcal{K}_t$ out to the action expert and to a newly initialized language-generation state; lines 5–13 then perform continuous batched decoding across all m active requests, after which finished requests are evicted and the rest are persisted for the next frame.



(a) ALOHA.



(b) DROID.

Figure 9 | Action-frequency vs. language-throughput tradeoff on the ALOHA and DROID configurations, sweeping total decoding steps N and decoding steps per frame k . Trends mirror the LIBERO results in Fig. 4: OxyGen extends the Pareto frontier beyond what either baseline can reach.

B. Additional Experimental Results

This section collects the experimental details and results omitted from Section 4: additional end-to-end speed results, per-suite action-quality numbers, and memory and energy measurements.

B.1. Additional End-to-End Results

The main text focuses on the LIBERO configuration; here we report the corresponding ALOHA and DROID tradeoff curves, a complementary speedup heatmap that varies total decoding steps, and the raw latency-throughput curves underlying the tradeoff plots.

B.1.1. End-to-End Tradeoff on ALOHA and DROID

Fig. 9 reports the same action-frequency / language-throughput tradeoff as Fig. 4 (which uses the LIBERO configuration) on the ALOHA and DROID configurations. OxyGen pushes the Pareto frontier outward on all three configurations, confirming that the conclusions in Section 4.2 are not specific to LIBERO.

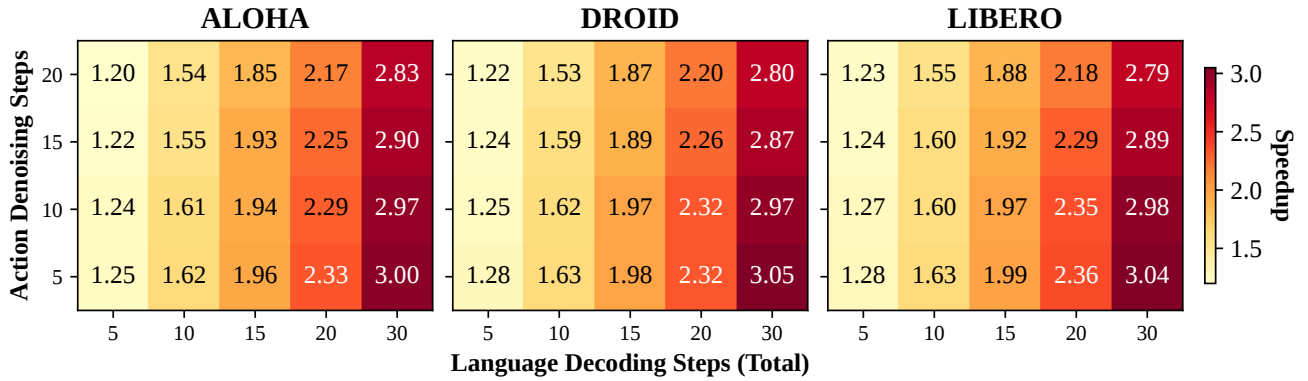


Figure 10 | Speedup ratio for action frequency of OxyGen vs. baseline across LIBERO, ALOHA, and DROID configurations (subplots), action denoising steps S (y-axis), and total decoding steps N (x-axis), with decoding steps per frame fixed at $k = 5$. The speedup increases with larger N , while the configuration choice and S have smaller impact.

Table 1 | Task success rate (%) on LIBERO test suites. Our results match the performance reported by openpi within statistical noise.

Setting	LIBERO-Spatial	LIBERO-Long	LIBERO-Goal	LIBERO-10
Reported (openpi)	98.8%	98.2%	98.0%	92.4%
Tested (ours)	98.0%	98.6%	97.4%	93.2%

B.1.2. Speedup Heatmap Across Total Decoding Steps

Fig. 10 complements the per-frame sweep in Fig. 5 by varying total decoding steps N at fixed steps per frame $k = 5$.

B.1.3. Raw Latency–Throughput Curves

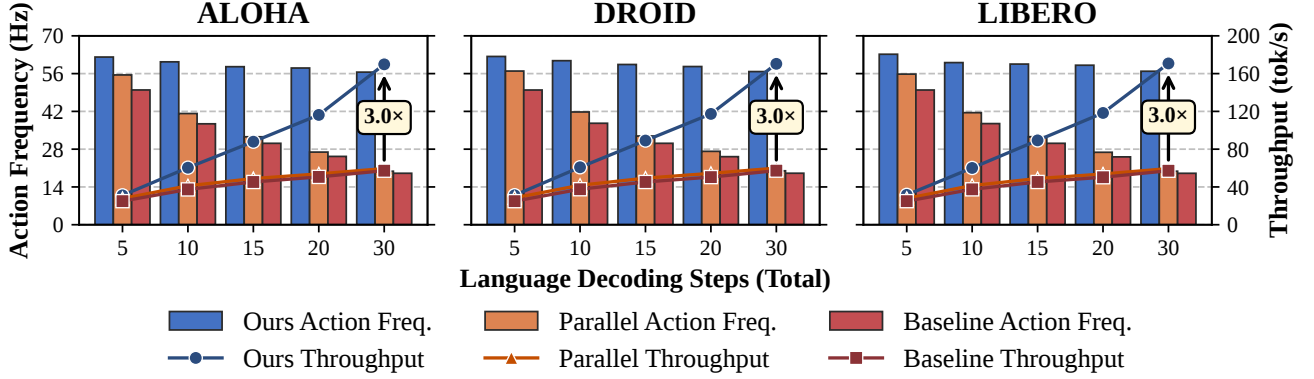
Fig. 11 reports raw action frequency and language throughput across settings, providing the underlying curves behind the tradeoff plots in Figs. 4 and 9.

B.2. Additional Action-Quality Results

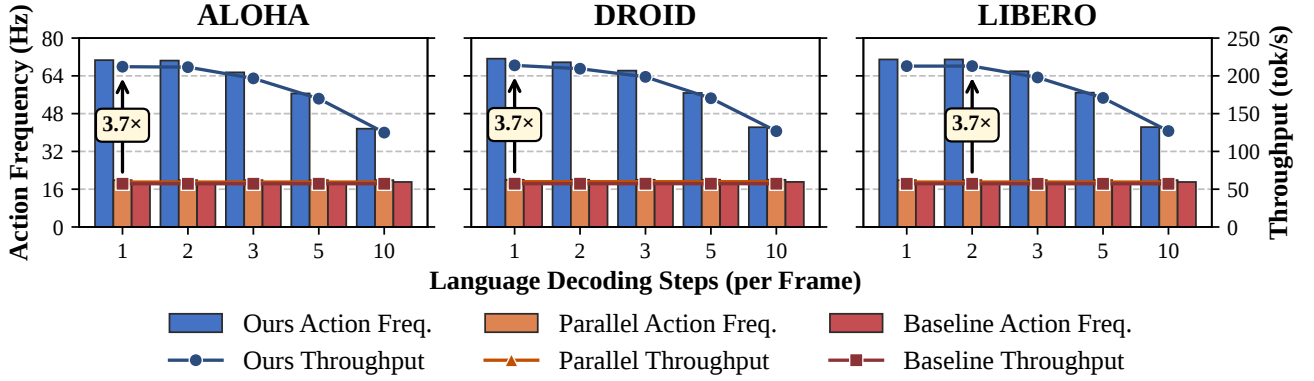
Table 1 reports the per-suite task success rates supporting the claim in Section 4.5 that OxyGen reproduces the openpi-reported numbers within statistical noise.

B.3. Additional Memory and Energy Measurements

Table 2 reports the memory and energy measurements supporting the headline numbers in Section 4.5.



(a) Varying total decoding steps (steps per frame $k = 5$).



(b) Varying steps per frame (total decoding steps $N = 30$).

Figure 11 | Raw action-frequency and language-throughput curves under different settings. The left plot varies total decoding steps N at fixed $k = 5$, and the right plot varies decoding steps per frame k at fixed $N = 30$. OxyGen consistently outperforms baselines by up to 3.7 $\times$, achieving up to 200 tokens/s language throughput and 70 Hz action frequency simultaneously.

Table 2 | Comparison of memory and energy cost. OxyGen adds modest memory overhead while achieving substantial energy savings.

Setting	Peak Mem. (GB)	Avg. Power (W)	Energy /Req. (mJ)	Δ Energy /Req. (%)
Baseline	6.43	293.5	117.4	0
Parallel	12.49	324.4	120.9	+3.0
Ours w/o batch	6.43	287.9	97.7	-16.8
Ours (batch: 2)	7.35	173.3	39.1	-66.7
Ours (batch: 4)	7.41	154.4	25.8	-78.0